

N-1 GitHub Action Security Scan and Remediation Pipeline Approach

Pipeline Overview

```
Stage 1: Checkout (clone the repo)
↓
Stage 2: Detect + Baseline (identify codebase type, install, build, test)
↓
Stage 3: Security Scan (Checkmarx + Black Duck against the repo)
↓
Stage 4: AI Analysis (normalize, deduplicate, rank scan results)
↓
Stage 5: Version Resolution (find nearest clean version per package)
↓
for each vulnerable package:
  Stage 6: Apply (save copy, update version, rebuild package list)
  ↓
  Stage 7: Verify (build, test, rescan)
  ↓
  Stage 8: Decide → fixed or needs manual review
  ↓
All packages done
↓
Stage 9: PR (one PR with all accepted fixes + report)
```

Stage 1: Checkout

Clone the repo at the target branch. Nothing is changed yet.

Stage 2: Detect + Baseline

Objective

Understand what kind of codebase this repo is and confirm that the build passes and all tests pass before we touch anything.

What is done

The pipeline inspects the repo to identify the codebase type — npm, pip, maven, nuget, cargo, or conda — based on which manifest and lockfile files are present. It then installs dependencies, runs the build, and runs the tests. If the build fails or any test fails, the pipeline stops. We do not proceed on a repo that is already broken.

Output

Confirmed codebase type, confirmed working build, confirmed passing tests. This is the baseline everything else is measured against.

Stage 3: Security Scan

Objective

Get a complete picture of what is vulnerable in the repo before making any changes.

What is done

Checkmarx and Black Duck are run against the checked-out repo. Both scanners examine the code and the full dependency tree — every package the application uses, direct and transitive — and produce a report each. Each report lists every package flagged as vulnerable, the version currently installed, and the CVE IDs that make it vulnerable.

Output

Two scanner reports. Together they give us the complete vulnerability list — every package that needs attention, its current version, and its CVEs. This is the full input to the rest of the pipeline. The scanners do not run again until Stage 7.

Stage 4: AI Analysis

Objective

Turn Checkmarx and Black Duck scanner reports in different formats into one clean, ordered list the pipeline can act on.

What is done

- **Normalize** — convert both reports into a single common structure: package name, current version, CVE IDs, direct or transitive.
- **Deduplicate** — the same package flagged for the same CVE by both scanners becomes one record.
- **Rank** — order by severity (critical → high → medium → low), then direct dependencies before transitive within the same severity.

The AI handles this stage because scanner report formats and schema versions change over time. Hard-coded parsers break when formats change. The AI reads both reports reliably regardless of format and produces the same structured output every time.

Output

A single ordered list. Each entry has the package name, current version, and the CVE IDs to close. The pipeline works through this list top to bottom.

Stage 5: Version Resolution

Objective

For each vulnerable package, find the nearest version that is not affected by the CVEs we need to close.

What is done

For each package in the list, two sources are queried:

- The package registry (npm, PyPI, Maven Central, NuGet, crates.io) — to get the full list of published versions.
- OSV/GHSA — to check which versions carry the CVEs and which are clean.

Starting from the version nearest to the current one, the pipeline asks: do any of the CVEs from Stage 4 apply to this version? The first version where the answer is no is the target. Same major version is preferred. If no clean version exists, the package is marked as unable to be fixed automatically and the pipeline moves to the next package.

Output

For each package: a target version to move to, or a note that it cannot be fixed automatically with the reason.

Stage 6: Apply

Objective

Apply the target version to the repo safely, with a restore point in place before any change is made.

```
Save a copy of the current package list and lockfile
↓
Update the vulnerable package to the target version
↓
Rebuild the package list to reflect the change
↓
New package list and lockfile ready for verification
```

What is done

Before touching anything, the current package list and lockfile are saved as a restore point in case Stage 7 fails. The target version from Stage 5 is then written into the package list and the lockfile is rebuilt using the repo's own package manager — npm, pip, mvn, dotnet, or cargo — without running any install scripts. No other files in the repo are touched.

Output

Updated package list and lockfile with the target version applied. Original copy saved so we can restore it if needed.

Stage 7: Verify

Objective

Prove that the change is safe — the CVEs are gone, nothing new is introduced, and the application still works.

What is done

The same three things run as in Stages 2 and 3, but now against the changed code:

- **Build** — the application must compile and build successfully with the new package version.
- **Test** — all tests that passed in Stage 2 must still pass.
- **Rescan** — Checkmarx and Black Duck run again. The CVEs we were trying to close must be gone. No new CVEs may appear that were not in the Stage 3 findings.

All of the following must be true: build passes, tests pass, the CVEs we were fixing are gone, and no new CVEs have been introduced. If any one of these fails, the change does not go forward.

Output

Pass or fail result with the full evidence — build log, test results, pre and post scan findings.

Stage 8: Decide

Objective

Determine whether the change from Stage 6 is safe to keep, based on the results from Stage 7.

```
Stage 7 result
├── Passed → keep the change → move to Stage 9
└── Failed → restore the package list to what it was
            ├── more candidate versions exist → try next version (back to Stage 6)
            └── no more candidates → record as needs manual review → next package
```

What is done

If Stage 7 passed — the change is accepted. The updated package list and lockfile stay in place.

If Stage 7 failed — the repo is restored to exactly what it was before Stage 6. The pipeline tries the next candidate version from Stage 5 and repeats Stages 6 and 7. If there are no more candidates, the package is recorded as unable to be fixed automatically and the pipeline moves on to the next package in the list.

Output

Either an accepted fix ready for Stage 9, or the repo restored to its original state with the package recorded for manual review.

Per-Package Loop

Each vulnerable package from Stage 5 goes through Apply → Verify → Decide independently.

```
Package 1: Apply → Verify → Decide → fixed or needs manual review
Package 2: Apply → Verify → Decide → fixed or needs manual review
Package 3: Apply → Verify → Decide → fixed or needs manual review

All packages done → one PR with all accepted fixes
```

Stage 9: PR

Objective

Deliver all accepted fixes to the developer in one pull request with full proof that each fix is safe.

What is done

A single pull request is opened containing all the package list and lockfile changes accepted in Stage 8. A report is attached listing every package fixed, the version it moved from and to, the CVEs closed, and the build, test, and scan results that proved each fix was safe. Packages that could not be fixed automatically are listed separately with the reason, so the developer knows what still needs manual attention.

Output

One pull request per pipeline run. The developer reviews the report, looks at the diff, and merges.